Software Requirements Specification

for

Financial Ledger & Wallet API

Version 1.0

Prepared by Vishudha Sood (IMT2024067), Shive S Bhat (BT2024067), Gourav Nayak (BT2024213)

International Institute of Information Technology Bangalore, CSE 303P Software Engineering Lab

9 November 2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Revision History ..... **ii**
1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 2
1.5 References 2
2. Overall Description **3**
2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Classes and Characteristics 4
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 User Documentation 5
2.7 Assumptions and Dependencies 5
3. External Interface Requirements **6**
3.1 User Interfaces 6
3.2 Hardware Interfaces 6
3.3 Software Interfaces 6
3.4 Communications Interfaces 7
4. System Features **8**
4.1 Wallet Initialization (FR-1) 8
4.2 Idempotent Transfers (FR-2) 9
4.3 Concurrent Balance Mutability (FR-3) 10
4.4 Immutable Audit Trail (FR-4) 11
4.5 Ledger Reconciliation (FR-5) 12
4.6 Cross-Currency Transfers (FR-6) 13
5. Other Nonfunctional Requirements **14**
5.1 Performance Requirements 14
5.2 Safety Requirements 14
5.3 Security Requirements 15
5.4 Software Quality Attributes 15
5.5 Business Rules 17
6. Other Requirements **17**
Appendix A: Glossary **19**
Appendix B: Analysis Models **21**
Appendix C: To Be Determined List **23**

Revision History

Name	Date	Reason For Changes	Version
Vishudha Sood, Shive S Bhat, Gourav Nayak	09-11-2026	First complete version, submitted for Stage 1 (Requirements)	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) specifies the requirements for release 1.0 of the Financial Ledger & Wallet API. The product is the term project of a three-member team in the CSE 303P Software Engineering Lab at IIIT Bangalore.

The SRS covers the whole product: the REST API, the double-entry ledger stored in SQLite, and the agent skills the team builds to develop, check and demonstrate the product. The product has no user interface of its own, so no user interface is specified.

1.2 Document Conventions

- **Requirement keywords.** Every requirement uses "shall" and is mandatory for the feature it belongs to. "May" marks behaviour that is permitted but not required. The words "should" and "must" are not used for requirements.
- **Identifiers.** FR-n.m are functional requirements. FR-1 to FR-5 keep the numbering of the course project statement, and FR-6 is added by the team. NFR-n are non-functional requirements, SK-n are agent skill requirements, TQ-n are testing requirements (TQ-1 to TQ-8 keep the course numbering), BR-n are business rules, DB-n are database requirements, C-n are design constraints and A-n are assumptions. Identifiers are never reused, so requirements can be traced to issues, pull requests, tests, skills and knowledge-base notes.
- **Priorities.** Each feature in Section 4 has a priority of High or Low, and its requirements inherit that priority. High-priority features are required for release 1.0. Low-priority features are implemented only after every High-priority feature passes its system tests (BR-9). Low-priority features are not required for release 1.0.
- **Money.** All amounts are integer minor units of the wallet currency (paise for INR, cents for USD and EUR). For example, ₹100.50 is written 10050. Field names that hold money end in `_minor`.
- **Typography.** Endpoints, HTTP headers, JSON fields, error codes and SQL keywords appear in `monospace`.
- **Terms.** Each term has one meaning, defined in Appendix A. In particular, "transaction" always means a ledger transaction (one money movement). A database transaction is always written "database transaction".

1.3 Intended Audience and Reading Suggestions

This SRS is written for the following readers:

Reader	Use of this SRS	Suggested sections
Development team (Ledger, API and Quality owners)	Build and test the product against the requirements	All sections, starting with 1.4, 2, 4 and Appendix B
Course instructor and teaching assistants	Review and grade the requirements	1.4, 2.2, 4, 5 and 6

Reader	Use of this SRS	Suggested sections
Testers (the team members, writing TQ-1 to TQ-13 tests)	Derive test cases from each requirement	4, 5, 6 and Appendix A
AI coding assistants used by the team	Read the SRS through the brain-query skill when drafting code, ADRs and tests	4, 5 and 6

Section 2 describes the product as a whole. Section 3 defines its external interfaces. Section 4 lists the functional requirements by feature. Section 5 lists the non-functional requirements, including the trade-offs between competing requirements (5.4). Section 6 lists the database, agent skill and testing requirements. The appendices hold the glossary, analysis models and the list of open items.

1.4 Product Scope

In a poorly designed wallet backend, two simultaneous updates can overwrite each other and a single network retry can charge a customer twice. The Financial Ledger & Wallet API is a backend service for a digital wallet in which every balance can be proved from an unchangeable history of money movements.

Its objectives are:

- every money movement is recorded as balanced double-entry ledger entries;
- a retried request never moves money twice;
- simultaneous requests on the same wallet never overspend it or lose an update;
- recorded history is never edited or deleted, and any direct tampering with it is detected;
- an administrator can verify at any time that the ledger is consistent.

In scope for release 1.0: accounts; wallets in several currencies; deposits; same-currency transfers; cross-currency transfers at administrator-set rates (Low priority, not required for release 1.0); reversals; ledger history; reconciliation; and the agent skills listed in Section 6.

Out of scope for release 1.0: any graphical user interface; connections to banks, cards or other real payment systems; withdrawals to external accounts; live exchange-rate feeds; per-user login (username and password with access tokens), which may be added in a later release; and deployment on a public network. No requirement in this document depends on an out-of-scope item.

1.5 References

- "Project Statement: Financial Ledger & Digital Wallet API" (Project_2_Financial_Ledger.pdf), CSE 303P course material, IIIT Bangalore LMS, 18 October 2026.
- "The AI-Assisted Software Development Life Cycle (SDLC)" (SDLC_and_Workflow.pdf), CSE 303P course material, IIIT Bangalore LMS, 18 October 2026.
- "Financial Ledger & Wallet API: 3-Week Team Plan", project team, 4 November 2026, in the project repository.
- IEEE Std 830-1998, "IEEE Recommended Practice for Software Requirements Specifications", IEEE, 1998.
- K. E. Wiegers, "Software Requirements Specification template", 1999 (the template this document follows).

- SQLite documentation: "Transaction" (https://www.sqlite.org/lang_transaction.html) and "Write-Ahead Logging" (https://www.sqlite.org/wal.html).
- RFC 9110, "HTTP Semantics", IETF, June 2022.
- ISO 4217, "Codes for the representation of currencies", International Organization for Standardization.
- FIPS 180-4, "Secure Hash Standard (SHS)", NIST, August 2015 (SHA-256).
- OpenAPI Specification 3.1.0, OpenAPI Initiative (https://spec.openapis.org/oas/v3.1.0).
- Agent Skills format specification (https://agentskills.io).

2. Overall Description

2.1 Product Perspective

The Financial Ledger & Wallet API is a new, self-contained product. It does not replace an existing system and is not part of a larger one. It runs as one server process with one SQLite database file. Clients, agent skills and the administrator all use it through the same HTTP/JSON interface. Figure 1 shows the product and its external interfaces.

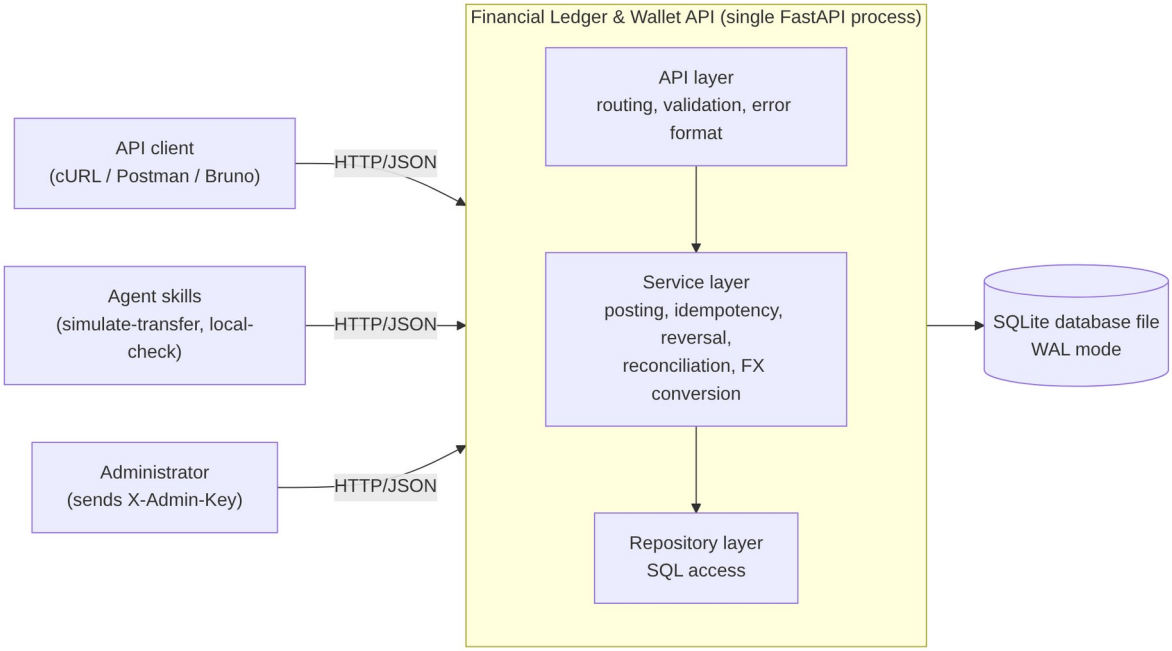


Figure 1. Product context and internal layers

The API layer validates requests and formats responses. The service layer applies all money rules. The repository layer is the only code that issues SQL. Each operation that moves money runs as one database transaction, so it is applied completely or not at all.

2.2 Product Functions

The product shall let its users:

- create accounts and one wallet per currency for each account (FR-1);

- deposit funds into a wallet through balanced ledger entries (FR-1);
- transfer funds between two wallets of the same currency, safely retrying with an idempotency key (FR-2);
- process simultaneous requests on the same wallet without overspending or lost updates (FR-3);
- read a wallet's balance and its ledger history, and reverse a transaction without changing history (FR-4);
- reconcile the ledger and detect tampering (FR-5);
- transfer funds between wallets of different currencies at administrator-set rates (FR-6, Low priority).

The development team shall also be able to run the agent skills in Section 6 to maintain the knowledge base, check a local installation and simulate a complete transfer workflow.

2.3 User Classes and Characteristics

User class	Characteristics	Functions used	Importance
API client	A program or developer calling the API with an HTTP tool (cURL, Postman, Bruno) or code. Technically skilled. Not authenticated in release 1.0.	Accounts, wallets, deposits, transfers, balances, history, exchange-rate list	Primary
Administrator	A team member operating the service who holds their own administrator key. Uses the same HTTP tools.	Reconciliation, reversals, setting exchange rates	Primary
Agent skill run by an AI assistant	Runs a scripted workflow (local-check, simulate-transfer) through the public API on behalf of a team member.	Same as API client; reconciliation through the team member's own administrator key	Secondary

2.4 Operating Environment

- Python 3.11 or later, with the SQLite library bundled with it (SQLite 3.35 or later).
- Developer machines of the three team members running Windows 10 or 11, macOS 13 or later, or Ubuntu 22.04 or later.
- GitHub Actions `ubuntu-latest` runners for continuous integration.
- One server process (Uvicorn) listening on 127.0.0.1, with the database file on a local disk. A network file share is not supported, because SQLite WAL mode requires a local file system.

2.5 Design and Implementation Constraints

C-1: The product shall be implemented in Python with the FastAPI framework, Pydantic for request validation and pytest for tests (team decision recorded in the team plan).

C-2: SQLite shall be the only database. No database server, cache server or distributed lock service shall be used.

C-3: Money shall be represented only as integers in minor units, in API fields, in program variables and in database columns. Floating-point values shall not be used for money anywhere.

C-4: External dependencies shall be isolated behind provider interfaces: database access behind a repository interface, and exchange-rate lookup (FR-6) behind a rate-provider interface (course requirement SE-6).

C-5: All work shall follow the course SDLC: GitHub Issues for all work and defects, a pull request with at least one review for every change, and a CI pipeline on every pull request that builds, lints, runs the tests, scans for secrets and validates every SKILL.md file and brain note.

C-6: Secrets (administrator keys) shall be held by clients only in environment variables, and by the server only as hashes. They shall never be committed to the repository or pasted into an AI tool.

C-7: Release 1.0 shall be tagged by 25 November 2026 by a team of three working part-time. This limits scope to the features in Section 4.

2.6 User Documentation

- **README.md:** installation, configuration (environment variables), how to start the server and how to run the tests.
- **API reference:** an OpenAPI 3.1 document generated by FastAPI, served at `/docs` and committed to the repository as `openapi.json`.
- **User guide:** one worked cURL example for every endpoint, including the error cases of Section 3.4.
- **Request collection:** a Postman or Bruno collection that runs the demonstration workflow.
- **Developer documentation:** architecture and design documents with UML and ER diagrams, and Architecture Decision Records in `docs/adr/`.
- **Skill documentation:** a SKILL.md file for each agent skill (Section 6).

2.7 Assumptions and Dependencies

A-1: The server is reachable only from the machine it runs on, so the absence of per-user login (Section 1.4) does not expose wallets to other people. If this assumption changes, per-user authentication becomes necessary.

A-2: Clients create a new, unique idempotency key (for example a UUID version 4) for each operation they intend, and reuse it only when retrying that same operation.

A-3: Exchange rates are entered by the administrator and are not required to match market rates.

A-4: The server clock is used only for the `created_at` timestamps. The order of transactions is defined by their sequence numbers, not by timestamps.

A-5: The product depends on these third-party components: FastAPI, Pydantic, Uvicorn, pytest, HTTPX (for tests), a Python linter, a secret scanner, GitHub and GitHub Actions, and Obsidian for the knowledge base. An incompatible change in any of them could require rework.

3. External Interface Requirements

3.1 User Interfaces

The product has no graphical user interface. Users interact with it through HTTP tools (cURL, Postman, Bruno) and through the interactive API page that FastAPI generates at `/docs`. Every error response shall follow the format in Section 3.4, so that a user can see which request field or rule caused the error.

3.2 Hardware Interfaces

None. The product does not interact with hardware directly. It uses only the operating system's file input and output for the database file.

3.3 Software Interfaces

SQLite (through the Python `sqlite3` module). On every database connection the product shall set `PRAGMA journal_mode=WAL`, `PRAGMA foreign_keys=ON`, `PRAGMA synchronous=FULL`, `PRAGMA fullfsync=ON` (which takes effect only on macOS) and a busy timeout of 5000 ms. Every operation that moves money shall start its database transaction with `BEGIN IMMEDIATE`.

Environment variables. `LEDGER_DB_PATH` gives the database file location. `LEDGER_ADMIN_KEY` holds an administrator's own key on the client side, for skills and scripts that call administrator endpoints (SK-6). The server shall refuse to start if `LEDGER_DB_PATH` is unset.

HTTP API. The product offers the following endpoints. All request and response bodies are JSON.

Method and path	Purpose	Headers required	Req.
`GET /health`	Report that the server and database are reachable (read-only)	none	SK-5
`POST /accounts`	Create an account	none	FR-1
`POST /accounts/{account_id}/wallets`	Create a wallet in a currency	none	FR-1
`GET /wallets/{wallet_id}`	Read a wallet and its balance	none	FR-1
`POST /wallets/{wallet_id}/deposits`	Deposit funds into a wallet	`Idempotency-Key`	FR-1
`POST /transfers`	Transfer funds between two wallets	`Idempotency-Key`	FR-2, FR-3, FR-6
`GET /transactions/{transaction_id}`	Read a transaction and its entries	none	FR-4
`GET /wallets/{wallet_id}/entries`	Read a wallet's ledger history	none	FR-4

Method and path	Purpose	Headers required	Req.
`POST /transactions/{transaction_id}/reversal`	Reverse a transaction	`Idempotency-Key`, `X-Admin-Key`	FR-4
`GET /admin/reconciliation`	Reconcile the ledger	`X-Admin-Key`	FR-5
`PUT /admin/fx-rates/{base}/{quote}`	Set an exchange rate	`X-Admin-Key`	FR-6
`GET /fx-rates`	List the active exchange rates	none	FR-6

The exact request and response schemas are defined in the OpenAPI document (Section 2.6). If a schema in the OpenAPI document conflicts with this SRS, this SRS takes precedence.

3.4 Communications Interfaces

- The product shall communicate over HTTP/1.1 (RFC 9110) on 127.0.0.1, port 8000 by default. TLS is not required in release 1.0 because of assumption A-1 (see trade-off T-6 in Section 5.4).
- Bodies shall be JSON encoded in UTF-8 with the `Content-Type: application/json` header.
- `Idempotency-Key` header: 8 to 128 characters, each a letter, digit, hyphen or underscore. `X-Admin-Key` header: the administrator key.
- A replayed response (FR-2.4) shall carry the header `Idempotent-Replayed: true`.
- Every error response shall have the body `{"error": {"code": "<CODE>", "message": "<text>"}}`, where `<CODE>` is one of the codes below and `<text>` names the field or rule that failed.

HTTP status	Error codes	Meaning
400 Bad Request	`VALIDATION_ERROR`, `IDEMPOTENCY_KEY_REQUIRED`, `SAME_WALLET`	The request is malformed or breaks an input rule
401 Unauthorized	`UNAUTHORIZED`	The administrator key is missing or wrong
404 Not Found	`ACCOUNT_NOT_FOUND`, `WALLET_NOT_FOUND`, `TRANSACTION_NOT_FOUND`	A referenced resource does not exist
409 Conflict	`WALLET_EXISTS`, `IDEMPOTENCY_KEY_REUSED`, `ALREADY_REVERSED`	The request conflicts with existing state
422 Unprocessable Content	`INSUFFICIENT_FUNDS`, `CURRENCY_MISMATCH`, `FX_RATE_NOT_FOUND`, `AMOUNT_TOO_SMALL`, `SYSTEM_WALLET_NOT_ALLOWED`, `CANNOT_REVERSE_REVERSAL`	The request is valid but a business rule rejects it
503 Service Unavailable	`SERVICE_BUSY`,	The database write lock was not

HTTP status	Error codes	Meaning
	`SERVICE_UNAVAILABLE`	obtained within the busy timeout; the request can be retried with the same idempotency key. `SERVICE_UNAVAILABLE`: `GET /health` could not read the database

4. System Features

This section organises the functional requirements by feature. FR-1 to FR-5 come from the course project statement. FR-6 is added by the team.

4.1 Wallet Initialization (FR-1)

4.1.1 Description and Priority

Accounts hold one wallet per currency. Every wallet balance comes only from double-entry ledger entries. Funds enter the system through deposits, which debit the system wallet of the currency and credit the user's wallet.

Priority: High. This is the base that every other feature uses.

4.1.2 Stimulus/Response Sequences

#	Stimulus	Response
1	Client sends `POST /accounts` with `{"name": "Asha"}`.	201 Created with the new account's `id`, `name` and `created_at`.
2	Client sends `POST /accounts/{id}/wallets` with `{"currency": "INR"}`.	201 Created with a wallet whose `balance_minor` is 0. If the account already has an INR wallet: 409 `WALLET_EXISTS`.
3	Client sends `POST /wallets/{id}/deposits` with an `Idempotency-Key` and `{"amount_minor": 50000}`.	201 Created with a DEPOSIT transaction: the INR system wallet is debited 50000 and the user wallet is credited 50000.
4	Client sends `GET /wallets/{id}`.	200 OK with the wallet, including `balance_minor: 50000`.

4.1.3 Functional Requirements

FR-1.1: The system shall create an account from a `name` of 1 to 100 characters and return a unique account ID (UUID version 4).

FR-1.2: The system shall create a wallet for an existing account in a supported currency (BR-3), with an initial balance of 0.

FR-1.3: The system shall allow at most one wallet per account per currency, and shall reject a second wallet in the same currency with 409 `WALLET_EXISTS`.

FR-1.4: When the database is created, the system shall create exactly one system wallet and one FX clearing wallet for each supported currency. These wallets belong to no account.

FR-1.5: The system shall record a deposit as one DEPOSIT transaction with two entries of equal amount: a debit to the system wallet of the currency and a credit to the user wallet.

FR-1.6: A deposit shall require an `Idempotency-Key` header and shall follow the idempotency rules FR-2.1 to FR-2.6.

FR-1.7: The system shall reject a deposit into a system wallet or an FX clearing wallet with 422 `SYSTEM_WALLET_NOT_ALLOWED`.

FR-1.8: The system shall change a wallet balance only by posting ledger entries. No endpoint shall set a balance directly.

FR-1.9: `GET /wallets/{wallet_id}` shall return the wallet's `id`, `account_id`, `currency`, `balance_minor` and `created_at`. The returned balance shall equal the sum of the wallet's credit entries minus the sum of its debit entries.

FR-1.10: The system shall respond 400 `VALIDATION_ERROR` to an ID that is not a valid UUID, and 404 to a valid ID that does not exist.

4.2 Idempotent Transfers (FR-2)

4.2.1 Description and Priority

A client moves funds from one wallet to another. Each transfer request carries an idempotency key, so the client can safely retry after a timeout or dropped connection: a retry returns the first result and moves no more money.

Priority: High.

4.2.2 Stimulus/Response Sequences

#	Stimulus	Response
1	Client sends `POST /transfers` with `Idempotency-Key: K` and `{"source_wallet_id": S, "destination_wallet_id": D, "amount_minor": 10000}`.	201 Created with the TRANSFER transaction and its two entries.
2	Client repeats the same request with the same key K.	The same status and body as the first response, with header `Idempotent-Replayed: true`. No new entries are written.
3	Client sends a different body with the same key K.	409 `IDEMPOTENCY_KEY_REUSED`. Nothing is written.
4	Client sends the request without an `Idempotency-Key`.	400 `IDEMPOTENCY_KEY_REQUIRED`. Nothing is written.
5	Client requests more than the source balance.	422 `INSUFFICIENT_FUNDS`. No ledger entries are written; the key and the 422 response are stored.

4.2.3 Functional Requirements

FR-2.1: Every endpoint that requires an `Idempotency-Key` header (deposits, transfers and reversals) shall reject a missing key with 400 `IDEMPOTENCY_KEY_REQUIRED` and a key not in the format of Section 3.4 with 400 `VALIDATION_ERROR`.

FR-2.2: The system shall compute a request fingerprint as the SHA-256 hash of the HTTP method, the request path and the request body serialised as JSON with sorted keys.

FR-2.3: For each request that reaches its business checks (a success, a 404, 409 `ALREADY_REVERSED` or a 422 response), the system shall store the key, fingerprint, response status and response body in the same database transaction as those checks and any ledger entries, so that either all of them are stored or none.

FR-2.4: When a request arrives with a stored key and the same fingerprint, the system shall return the stored status and body without writing any ledger entries. The key lookup shall take place inside the database transaction of FR-3.1.

FR-2.5: When a request arrives with a stored key and a different fingerprint, the system shall respond 409 `IDEMPOTENCY_KEY_REUSED` without writing anything.

FR-2.6: A request rejected with 400, 401, 409 `IDEMPOTENCY_KEY_REUSED`, 503 or 500 shall not store its key, so a later request with that key is processed as a new request. Stored keys shall be kept permanently.

FR-2.7: The system shall accept `amount_minor` only as a JSON integer from 1 to the maximum in BR-4. A decimal number, string, zero or negative value shall be rejected with 400 `VALIDATION_ERROR`.

FR-2.8: The system shall reject a transfer whose source and destination are the same wallet with 400 `SAME_WALLET`, and a transfer to or from a system or FX clearing wallet with 422 `SYSTEM_WALLET_NOT_ALLOWED`.

FR-2.9: The system shall reject a transfer with 422 `INSUFFICIENT_FUNDS` when the source wallet balance is less than the amount.

FR-2.10: For wallets of the same currency, the system shall record one TRANSFER transaction with a debit to the source wallet and a credit to the destination wallet of the same amount.

FR-2.11: Until FR-6 is implemented, the system shall reject a transfer between wallets of different currencies with 422 `CURRENCY_MISMATCH` and write no ledger entries. Once FR-6 is implemented, the system shall process such a transfer as specified in FR-6.

FR-2.12: The success response shall contain the transaction ID, its type, its entries, and the new balances of the source and destination wallets.

4.3 Concurrent Balance Mutability (FR-3)

4.3.1 Description and Priority

Several requests may change the same wallet at the same moment. The system serialises all operations that move money, so no two of them can both spend the same funds and no update is lost.

Priority: High.

4.3.2 Stimulus/Response Sequences

#	Stimulus	Response
1	20 clients send transfers of 10000 from wallet S (balance 50000) at the same moment, each with its own key.	Exactly 5 transfers return 201 and 15 return 422 `INSUFFICIENT_FUNDS`. The final balance of S is 0.
2	A request cannot obtain the write lock within the busy timeout.	503 `SERVICE_BUSY`. Nothing is written, and the client may retry with

#	Stimulus	Response
		the same key.

4.3.3 Functional Requirements

FR-3.1: Every deposit, transfer and reversal shall run in one database transaction started with `BEGIN IMMEDIATE`, so that the write lock is held before any balance is read.

FR-3.2: The balance check, the ledger entries, the balance updates, the hash-chain update and the idempotency record of one operation shall all happen inside that one database transaction.

FR-3.3: If any step inside the database transaction fails, the system shall roll back the whole database transaction, so no partial operation remains.

FR-3.4: A request that does not obtain the write lock within the 5000 ms busy timeout shall receive 503 `SERVICE_BUSY` with nothing written.

FR-3.5: For a user wallet with balance B and N simultaneous transfers of amount a each, where $N \times a > B$, exactly $\lfloor B \div a \rfloor$ transfers shall succeed, the others shall receive 422 `INSUFFICIENT_FUNDS`, and the final balance shall be $B - \lfloor B \div a \rfloor \times a$.

FR-3.6: The database shall reject, through a `CHECK` constraint, any balance below 0 for a user wallet, as a final safeguard against a defect in application code.

4.4 Immutable Audit Trail (FR-4)

4.4.1 Description and Priority

Once written, transactions and ledger entries never change. A mistake is corrected by a new REVERSAL transaction whose entries mirror the original. Each transaction stores a hash that links it to the previous one, so a direct change to the database file can be detected (FR-5).

Priority: High.

4.4.2 Stimulus/Response Sequences

#	Stimulus	Response
1	Administrator sends `POST /transactions/{id}/reversal` with an `Idempotency-Key`, the `X-Admin-Key` and a `reason` in the body.	201 Created with a REVERSAL transaction that debits each wallet the original credited and credits each wallet the original debited, by the same amounts. The original transaction is unchanged.
2	Administrator reverses the same transaction again with a new key.	409 `ALREADY_REVERSED`.
3	Code or a user tries to `UPDATE` or `DELETE` a ledger row through SQL.	The database aborts the statement.
4	Client sends `GET /wallets/{id}/entries`.	200 OK with the wallet's entries in sequence order.

4.4.3 Functional Requirements

FR-4.1: Database triggers shall abort every `UPDATE` and `DELETE` on the `transactions`, `ledger_entries`, `idempotency_keys` and `fx_rates` tables.

FR-4.2: No endpoint shall modify or delete a transaction or a ledger entry.
FR-4.3: The reversal endpoint shall require the administrator key (NFR-9) and an `Idempotency-Key`, and shall create one REVERSAL transaction that references the original and mirrors each of its entries with the opposite direction and the same wallet and amount. The request body shall contain a `reason` of 1 to 200 characters, and the REVERSAL transaction shall record it together with the administrator who made the reversal (NFR-9). The reversal endpoint shall follow the idempotency rules FR-2.1 to FR-2.6.
FR-4.4: A transaction shall be reversed at most once (409 `ALREADY_REVERSED`), and a REVERSAL transaction shall not be reversed (422 `CANNOT_REVERSE_REVERSAL`).
FR-4.5: A reversal that would make a user wallet balance negative shall be rejected with 422 `INSUFFICIENT_FUNDS`, with no ledger entries written.
FR-4.6: Each transaction shall have a sequence number one greater than the previous transaction, starting at 1.
FR-4.7: Each transaction shall store `hash` = SHA-256(`prev_hash` ‖ canonical JSON of the transaction and its entries), where `prev_hash` is the hash of the previous transaction, or 64 zero characters for the first. The canonical JSON shall hold the transaction's `id`, `seq`, `type`, `reverses_id`, `fx_rate_id`, `admin_id`, `reason` and `created_at`, and its entries in entry-id order, each with `wallet_id`, `direction`, `amount_minor` and `balance_after`, serialised with sorted keys, no whitespace and UTF-8 encoding.
FR-4.8: `GET /wallets/{wallet_id}/entries` shall return the wallet's entries in sequence order, each with transaction ID, transaction type, direction, amount, balance after the entry and timestamp. It shall return 50 entries per page by default and at most 500. Clients shall page with the query parameters `limit` (1 to 500, default 50) and `after_id`. A `limit` outside that range shall be rejected with 400 `VALIDATION_ERROR`. Each page shall include `next_after_id`, which is null on the last page.
FR-4.9: `GET /transactions/{transaction_id}` shall return the transaction, its entries, and the ID of the transaction it reverses or that reverses it, if any.

4.5 Ledger Reconciliation (FR-5)

4.5.1 Description and Priority

An administrator runs a read-only check of the whole ledger. The check proves that debits equal credits, that stored balances match the entries, and that the hash chain is unbroken.

Priority: High.

4.5.2 Stimulus/Response Sequences

#	Stimulus	Response
1	Administrator sends `GET /admin/reconciliation` with the correct `X-Admin-Key`.	200 OK with a report whose `status` is `PASS` and whose checks all pass.
2	A ledger row was changed directly in the database file, bypassing the API.	200 OK with `status: FAIL`, naming the first transaction whose hash does not match.
3	The request has no `X-Admin-Key` or a wrong one.	401 `UNAUTHORIZED`.

4.5.3 Functional Requirements

FR-5.1: The reconciliation endpoint shall require the administrator key (NFR-9).

FR-5.2: The system shall check that: (a) within each transaction, debits equal credits for each currency; (b) across the whole ledger, total debits equal total credits for each currency; (c) each wallet's stored balance equals its credits minus its debits; (d) no user wallet has a negative balance; (e) every transaction has at least two entries; (f) sequence numbers have no gaps; and (g) every stored hash equals the hash recomputed under FR-4.7.

FR-5.3: The response shall contain an overall `status` (`PASS` or `FAIL`), the result of each check, every discrepancy found (with the wallet or transaction ID, the expected value and the actual value), the total debits and credits per currency, the number of transactions checked and the time of the check. It shall also contain the sequence number and hash of the latest transaction, so that a later report can show whether transactions were removed.

FR-5.4: Reconciliation shall be read-only and shall not change any data. It shall read all data within one read transaction, so that it checks one consistent snapshot.

FR-5.5: Reconciliation shall return 200 with `status: FAIL` when a check fails, so that the report itself is the result.

4.6 Cross-Currency Transfers (FR-6)

4.6.1 Description and Priority

A client transfers funds between wallets of different currencies. The administrator sets the exchange rates. The transfer passes through the FX clearing wallets, so that debits equal credits within each currency on its own.

Priority: Low. Implemented only after FR-1 to FR-5 pass their system tests (BR-9). Until then, FR-2.11 applies and cross-currency transfers are rejected.

4.6.2 Stimulus/Response Sequences

#	Stimulus	Response
1	Administrator sends `PUT /admin/fx-rates/USD/INR` with `{"rate_scaled": 88123456}` and the `X-Admin-Key`.	200 OK. The USD to INR rate is now 88.123456.
2	Client transfers `amount_minor: 1000` (US$10.00) from a USD wallet to an INR wallet.	201 Created. The INR wallet receives ⌊1000 × 88123456 ÷ 1,000,000⌋ = 88123 paise (₹881.23).
3	Client transfers between currencies that have no rate set.	422 `FX_RATE_NOT_FOUND`. No ledger entries are written; the key and the 422 response are stored.

4.6.3 Functional Requirements

FR-6.1: The system shall let an administrator set the rate for an ordered currency pair (base, quote) as a positive integer `rate_scaled`, equal to the rate multiplied by 1,000,000. Each change shall add a new rate record. The newest record for a pair is the active rate. Rate records shall never be

updated or deleted. `rate_scaled` shall be from 1 to 1,000,000,000,000. Each rate record shall record the administrator who set it.

FR-6.2: `GET /fx-rates` shall list the active rate of every pair that has one.

FR-6.3: The destination amount shall be ⌊source amount × `rate_scaled` ÷ 1,000,000⌋, computed with integer arithmetic only, using the active rate for (source currency, destination currency). The rate for the reverse pair shall not be inferred.

FR-6.4: The system shall reject a transfer with 422 `FX_RATE_NOT_FOUND` when the pair has no active rate, and with 422 `AMOUNT_TOO_SMALL` when the destination amount computes to 0.

FR-6.5: The system shall record one TRANSFER transaction with four entries: a debit to the source wallet and a credit to the FX clearing wallet in the source currency, and a debit to the FX clearing wallet and a credit to the destination wallet in the destination currency.

FR-6.6: The transaction shall record the rate record it used. The response shall include the destination amount and the `rate_scaled` used.

FR-6.7: A replayed request (FR-2.4) shall return the original result even if the rate has changed since. A reversal (FR-4.3) shall use the amounts in the original entries, not the current rate.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Correctness has priority over speed (Section 5.4). The figures below apply on a developer machine with at least 4 CPU cores, 8 GB of memory and an SSD, with a database holding 10,000 transactions.

NFR-1: Over 1,000 sequential transfer requests from one client, the 95th-percentile response time of `POST /transfers` shall be at most 100 ms.

NFR-2: Over 1,000 sequential requests, the 95th-percentile response time of `GET /wallets/{wallet_id}` shall be at most 50 ms.

NFR-3: 20 simultaneous transfer requests on the same wallet shall all receive a final response (success or rejection) within 10 seconds in total, with none receiving 503 `SERVICE_BUSY`.

NFR-4: Reconciliation of a ledger with 10,000 transactions shall complete within 5 seconds.

5.2 Safety Requirements

In this product, harm means losing money, creating money or losing records of it.

NFR-5: A transaction for which the client received a success response shall still be present after the server process is killed or the machine loses power (SQLite with `synchronous=FULL`).

NFR-6: If the server stops at any point during an operation, the database shall afterwards contain either all of that operation's effects or none of them.

NFR-7: Automated tests shall never use the database file given by `LEDGER_DB_PATH`. Each test run shall use its own new database (TQ-5).

NFR-8: The API shall reject a JSON number with a fractional part or exponent in any money field, and every money column shall have a `CHECK (typeof(column) = 'integer')` constraint.

5.3 Security Requirements

NFR-9: Administrator endpoints (reversal, reconciliation, exchange rates) shall respond 401 `UNAUTHORIZED` unless the `X-Admin-Key` header matches an active key in the `admin_keys` table. Each administrator shall have their own key of at least 32 random characters, created or revoked only through a command-line tool run on the server machine, which shows the key once. Only the SHA-256 hash of a key shall be stored, and a revoked key shall be rejected. Administrator authentication shall be checked before any idempotency lookup or other processing of the request.

NFR-10: No administrator key shall appear in a log, a response body, the repository or a stored idempotency record.

NFR-11: Every request parameter shall be validated against its type, format, length and range. Unknown JSON fields shall be rejected with 400 `VALIDATION_ERROR`, so negative or malformed values cannot be used to create money.

NFR-12: All SQL statements shall use parameter binding. Request data shall never be inserted into SQL text.

NFR-13: Error responses shall not contain stack traces, SQL text or file paths.

NFR-14: The server shall listen only on 127.0.0.1 unless the operator sets a different address explicitly.

NFR-15: The CI pipeline shall run a secret scanner on every pull request and shall fail when a secret is found.

5.4 Software Quality Attributes

NFR-16: **Maintainability.** Endpoint handlers shall contain no SQL and no money rules. Money rules shall be only in the service layer, and SQL only in the repository layer.

NFR-17: **Extensibility.** Access control shall be implemented in one replaceable FastAPI dependency, so that per-user authentication can be added later without changing the business logic of any endpoint.

NFR-18: **Testability.** Automated tests shall cover at least 85% of the lines in the service layer, measured in CI.

NFR-19: **Code quality.** The linter configured in the repository shall report zero errors on the main branch.

NFR-20: **Portability.** The local-check skill (SK-5) shall pass on the machine of every team member and on the CI runner.

NFR-21: **Usability of the API.** Every error shall use the format and codes in Section 3.4, and every endpoint shall appear in the OpenAPI document with an example request and response.

Competing requirements and their resolution. Some of the requirements above work against each other. Table 1 records each conflict and the decision the team took. Where two requirements compete, the order of preference is: correctness of money, then durability, then security, then performance.

ID	Competing requirements	Conflict	Resolution and rationale
T-1	Race safety (FR-3.1) vs.	`BEGIN IMMEDIATE` lets	Correctness wins. Database

ID	Competing requirements	Conflict	Resolution and rationale
	throughput and latency (NFR-1, NFR-3)	only one money-moving operation run at a time, so simultaneous requests queue instead of running in parallel.	transactions are kept short (no network calls or file I/O inside them). WAL mode lets reads continue during a write, so only writes queue. NFR-3 sets a limit the team can test.
T-2	Durability (NFR-5) vs. write latency (NFR-1)	`synchronous=FULL` waits for the disk on every commit. `NORMAL` is faster but can lose the last commits on power loss.	Durability wins: a client is never told money moved when the record could vanish. The 100 ms budget in NFR-1 allows for the disk wait.
T-3	Immutability (FR-4.1) vs. correcting errors and limiting storage	Wrong entries cannot be fixed in place, and the database only ever grows.	Corrections are made only through reversals (FR-4.3), which keep a complete audit trail. Storage growth is small at the scale of this project and is accepted.
T-4	Tamper evidence (FR-4.7, FR-5.2(g)) vs. parallel writes and reconciliation time (NFR-4)	Each hash depends on the previous one, so transactions are written strictly one after another, and checking the chain means re-reading the whole ledger.	Writes are already serialised by T-1, so the chain adds no extra waiting. Reconciliation is a periodic administrator task, so a full pass bounded by NFR-4 is acceptable.
T-5	Exact integer money (C-3, NFR-8) vs. fair currency conversion (FR-6.3)	Conversion produces fractions of a minor unit, which integers cannot hold.	The destination amount is rounded down. The fraction is not paid out, so the system can never create money. Each currency still balances on its own through the FX clearing wallets (FR-6.5). The rounding rule is stated in the API documentation.
T-6	Scope and simplicity (no per-user login, no TLS) vs. protection of funds (Section 5.3)	Without login, any client that can reach the server can move funds from any user wallet, or deposit any amount up to the BR-4 limit into any wallet.	Release 1.0 runs only on the local machine (A-1, NFR-14). Administrator actions are protected by per-administrator keys and attributed to the administrator who made them (NFR-9, FR-4.3). Access control is isolated (NFR-17) so login can be added later.
T-7	Retry safety (FR-2.4, FR-3.4) vs. permanent key storage and freedom to reuse keys (FR-2.6)	Keeping every key forever uses storage and blocks reuse, while deleting keys reopens the double-charge	Keys are kept permanently, so a late retry can never double-charge. Requests rejected before their

ID	Competing requirements	Conflict	Resolution and rationale
		risk.	business checks (400, 401, 503) do not store their key, so the client can retry after fixing them. Business rejections are stored, so a key always returns the same answer.

Table 1. Competing requirements and their resolution

5.5 Business Rules

BR-1: In every transaction, the total of debit entries shall equal the total of credit entries for each currency.
BR-2: Only system wallets and FX clearing wallets shall be allowed a negative balance.
BR-3: The supported currencies in release 1.0 are INR, USD and EUR, each with 2 decimal places (minor unit = 1/100).
BR-4: One deposit, transfer or reversal request shall carry an `amount_minor` from 1 to 100,000,000 (₹10,00,000.00 or the same number of cents). For a cross-currency transfer, this limit applies to the source amount.
BR-5: Only an administrator shall be able to reverse a transaction, run reconciliation or set exchange rates.
BR-6: Transfers shall be only between user wallets. System and FX clearing wallets take part only in deposits, cross-currency transfers and reversals of them.
BR-7: Accounts and wallets shall not be deleted, because ledger entries refer to them.
BR-8: A transaction shall be reversed at most once, and a reversal shall not be reversed.
BR-9: Work on a Low-priority feature (FR-6) shall start only after every High-priority feature (FR-1 to FR-5) passes its system tests.

6. Other Requirements

Database requirements

DB-1: The database shall contain the tables `accounts`, `wallets`, `transactions`, `ledger_entries`, `idempotency_keys`, `fx_rates` and `admin_keys`, as shown in Appendix B, Figure 2.
DB-2: Every money column shall be `INTEGER`, every ledger entry amount shall be greater than 0, and foreign keys shall be enforced.
DB-3: The schema shall be created and changed only through numbered migration files kept in the repository.
DB-4: Timestamps shall be stored in UTC in ISO 8601 format. Identifiers of accounts, wallets, transactions and rate records shall be UUID version 4 strings.

Agent skill requirements (SK)

SK-1: Each skill shall be a folder `skills/<name>/` containing a `SKILL.md` that follows the Agent Skills format, with YAML front matter holding `name` (lowercase words joined by hyphens, equal to the folder name) and `description`. CI shall reject a pull request in which any `SKILL.md` breaks this rule.

SK-2: **brain-ingest** shall turn new source material into a note in `brain/inbox/` using the research note template (front matter: type, title, tags, requirements, sources, author, created), and shall add a row to `brain/log.md`.

SK-3: **brain-refine** shall move notes from `brain/inbox/` to the correct PARA folder, merge duplicate notes, repair broken `[[wikilinks]]`, update `brain/index.md` and add a row to `brain/log.md`.

SK-4: **brain-query** shall answer a question or draft a document section (such as an SRS or ADR section) using only notes in the brain, and shall list every note it used as a `[[wikilink]]`.

SK-5: **local-check** shall: (1) install dependencies and build the project; (2) run the automated test suite; (3) start the local server; (4) call the read-only endpoint `GET /health`; and (5) report pass or fail for each of the four steps. **local-check** shall start the server with a temporary database file and an administrator key it creates with the command-line tool of NFR-9, so it needs no configuration and never uses the database named by `LEDGER_DB_PATH`. `GET /health` shall run one read-only database query and return 200 with `{"status": "ok"}`, or 503 `SERVICE_UNAVAILABLE` if the query fails.

SK-6: **simulate-transfer** shall: (1) create two test wallets in one currency and deposit a fixed amount into the first; (2) record their balances; (3) execute a transfer with a new idempotency key, repeat it with the same key, and check that only one transfer was recorded; (4) send two simultaneous transfers whose total exceeds the source balance; and (5) call reconciliation and summarise the final balances. It shall generate new idempotency keys on every run. It shall report failure if reconciliation returns `FAIL`, if both simultaneous transfers succeed, or if any final balance differs from the balance computed from the amounts it sent. It shall read the administrator key from the `LEDGER_ADMIN_KEY` environment variable inside its script, and shall never print the key or pass its value to the AI assistant (C-6).

SK-7: Every skill shall cite at least one distilled note from the brain.

SK-8: Every skill shall have written evaluation cases with expected results, run before each stage submission, with results recorded in the test documentation.

Testing requirements (TQ)

TQ-1: Unit tests shall cover the money utility (minor-unit handling and conversion), ledger balance assertions, request fingerprint hashing (FR-2.2) and the hash chain (FR-4.7).

TQ-2: Integration tests shall show that a failure injected at each step of a transfer leaves no entries, no balance change and no idempotency record (FR-3.3). They shall also check that, after every posting, each stored wallet balance equals the sum of that wallet's entries.

TQ-3: System (end-to-end) tests through the HTTP API shall cover each of FR-1 to FR-5, and FR-6 when it is implemented.

TQ-4: Negative tests shall cover negative and zero amounts, insufficient funds, a reused key with a different payload, malformed keys and IDs, and the simultaneous-transfer scenario of FR-3.5 with at least 20 requests.

TQ-5: CI shall run all tests on every push against a new, isolated database. Concurrency tests shall use a temporary database file, because each connection to an in-memory SQLite database gets its own separate database.

TQ-6: Tests shall show that no floating-point value reaches a money field, calculation or column (C-3, NFR-8).

TQ-7: Skill evaluations (SK-8) shall run before each stage submission, with results recorded in the test documentation.

TQ-8: Tests shall be updated as the system changes, and every significant defect shall get a regression test linked to its GitHub issue.

TQ-9: Security tests shall cover NFR-9 (missing, wrong and revoked administrator keys), NFR-11 (unknown fields, out-of-range values) and NFR-13 (no internal details in errors).

TQ-10: A tamper test shall disable the immutability triggers, change one ledger entry directly in the database file, and check that reconciliation returns `FAIL` naming that transaction (FR-5.2(g)).

TQ-11: Performance tests shall measure NFR-1 to NFR-4 on hardware meeting Section 5.1 before each stage submission, with results recorded in the test documentation. They shall not gate pull requests.

TQ-12: A crash test shall force-kill the server process after a transfer has received 201, restart it, and check that the transaction is present and that reconciliation returns `PASS` (NFR-5, NFR-6).

TQ-13: NFR-10, NFR-12, NFR-14, NFR-16, NFR-17 and NFR-21 shall be verified by inspection during pull-request review, and the power-loss part of NFR-5 by inspection of the database settings (Section 3.3).

Appendix A: Glossary

Term	Definition
Account	A record for one user of the wallet. It owns at most one wallet per currency.
ACID	Atomicity, consistency, isolation and durability: the guarantees of a database transaction.
ADR	Architecture Decision Record: a short document in `docs/adr/` recording one significant design decision and its reasons.
Administrator key	A secret issued to one administrator by the command-line tool of NFR-9 and sent in `X-Admin-Key` to use administrator endpoints. Only its SHA-256 hash is stored.
Agent skill	A folder with a `SKILL.md` file that tells an AI assistant how to perform one task.
`BEGIN IMMEDIATE`	An SQLite statement that starts a database transaction and takes the write lock at once.
Brain (second brain)	The project knowledge base: an Obsidian vault in `brain/` organised with PARA.
Busy timeout	How long SQLite waits for a lock before giving up (5000 ms in this product).
Credit	A ledger entry that adds money to a wallet.
Database transaction	A group of database statements that SQLite applies

Term	Definition
	completely or not at all. Not the same as a transaction.
Debit	A ledger entry that removes money from a wallet.
Double-entry	The accounting rule that each money movement is recorded as debits and credits of equal total.
FX clearing wallet	A wallet that belongs to no account, one per currency, through which cross-currency transfers pass (FR-6.5).
Hash chain	The sequence of transaction hashes in which each hash includes the previous one (FR-4.7).
Idempotency key	A client-chosen value sent in `Idempotency-Key` that identifies one intended operation, so retries of it have no further effect.
Ledger entry	One debit or credit of one amount to one wallet, belonging to one transaction.
Minor unit	The smallest unit of a currency: 1 paisa for INR, 1 cent for USD and EUR.
p95 (95th percentile)	The response time that 95% of measured requests do not exceed.
PARA	Projects, Areas, Resources, Archive: the folder scheme of the brain.
Reconciliation	The read-only check of the ledger defined in FR-5.
Request fingerprint	The SHA-256 hash of method, path and body that identifies the content of a request (FR-2.2).
Reversal	A REVERSAL transaction that cancels an earlier transaction by mirroring its entries.
System wallet	A wallet that belongs to no account, one per currency, that represents money entering from outside. Debited by deposits.
Transaction	One money movement (DEPOSIT, TRANSFER or REVERSAL) recorded as two or more ledger entries.
UUID	Universally unique identifier. This product uses version 4 (random) UUIDs.
Wallet	A balance in one currency. A user wallet belongs to one account.
WAL	Write-Ahead Logging: an SQLite mode that lets reads continue while a write is in progress.

Appendix B: Analysis Models

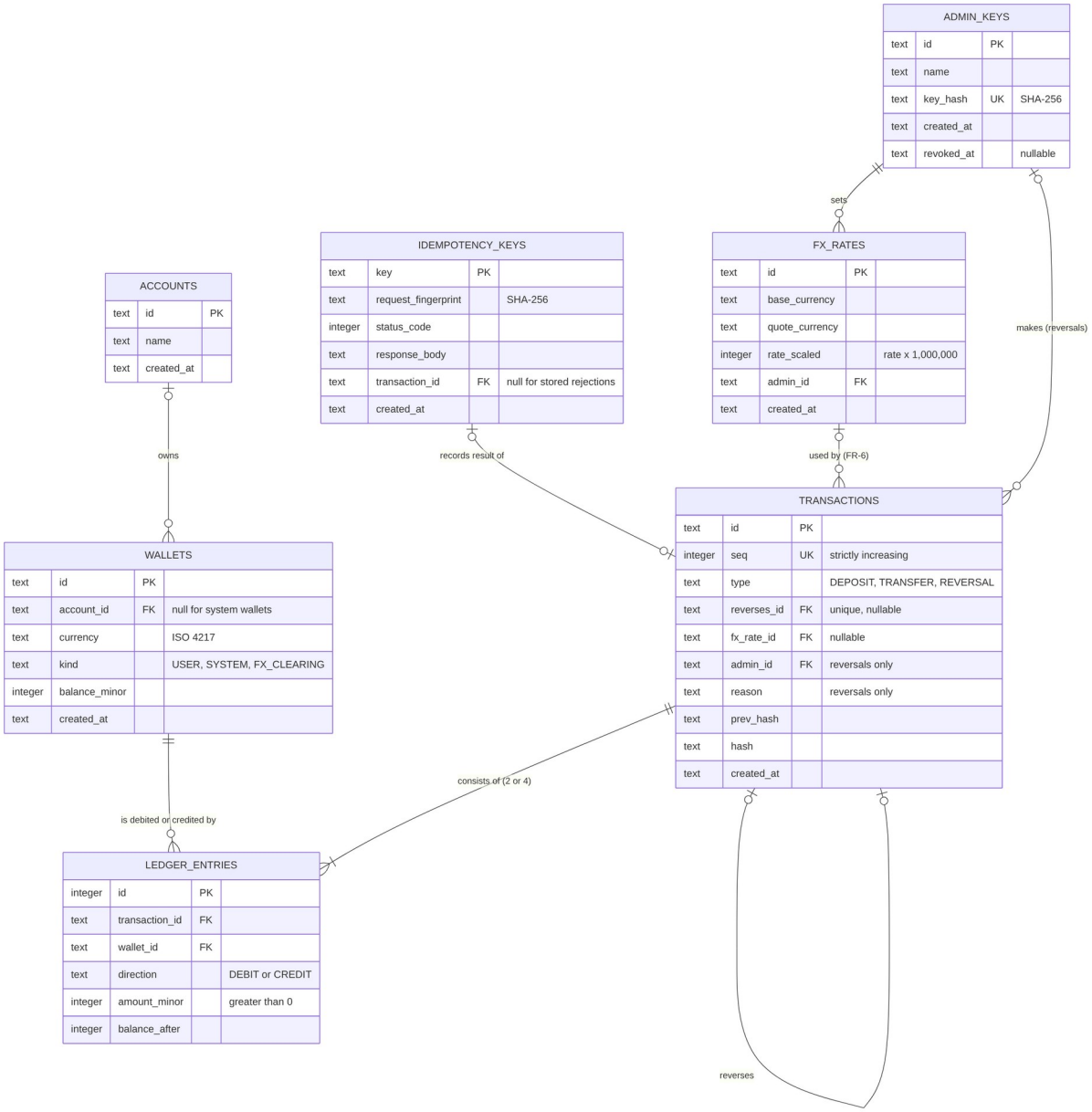


Figure 2. Entity-relationship model of the ledger database

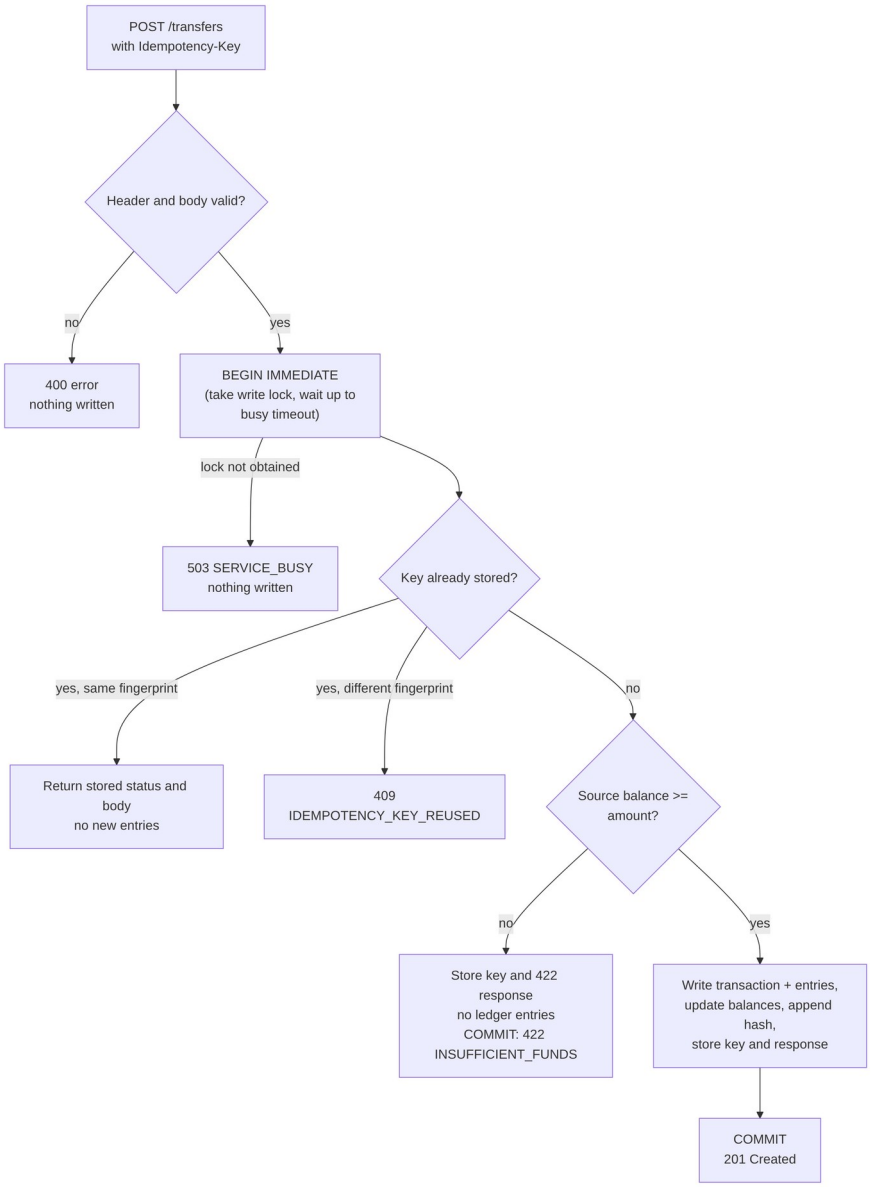


Figure 3. Processing of one transfer request (FR-2, FR-3)

Table 2 shows the entries recorded when Asha (₹500.00) transfers ₹100.00 to Ravi, and when the transfer is then reversed.

Seq.	Transaction	Wallet	Direction	amount_minor	Balance after
1	DEPOSIT	INR system wallet	Debit	50000	−50000
1	DEPOSIT	Asha (INR)	Credit	50000	50000
2	TRANSFER	Asha (INR)	Debit	10000	40000
2	TRANSFER	Ravi (INR)	Credit	10000	10000
3	REVERSAL of 2	Ravi (INR)	Debit	10000	0
3	REVERSAL of 2	Asha (INR)	Credit	10000	50000

Table 2. Example ledger entries for a deposit, a transfer and its reversal

Appendix C: To Be Determined List

No items are to be determined. Every value this SRS depends on is stated in it.